

MI FB API

1 Overview

The API of MStarFB is divided into the following 4 categories:

1.1. File operation class

Provides an interface for operating MStarFB. By calling these interfaces, overlays can be manipulated like files. These interfaces are standard interfaces provided by Linux itself, mainly including `open`, `close`, `write`, `read`, and so `lseek` on. These standard interfaces are not described in this document.

1.2. Video memory mapping class

Provides an interface for mapping physical video memory to user virtual memory space. These interfaces are standard interfaces provided by Linux itself, mainly including `mmap`, `munmap` etc. These standard interfaces are not described in this document.

1.3. Video memory control and status query class

An interface that allows setting properties such as pixel format and color depth. These interfaces are standard interfaces provided by Linux itself and are often used. This document will briefly describe it.

1.4. Interlayer effect control and state query class

MStarFB can manage multiple graphics overlays, and each layer can set properties such as alpha information and display area. These are new features of MStarFB relative to the Linux Framebuffer. This document will focus on that part. This part of the function is achieved by extending the command parameters of the standard linux Framebuffer ioctl function.

ioctl function:

The user mode interface of MStarFB is embodied in the form of ioctl, and its form is as follows:

```
int  ioctl ( int  fd ,
            unsigned long  cmd ,
            ...
        );
```

This function is a Linux standard interface with variable parameters. But in MStarFB, only 3 parameters are actually required. Therefore, its grammatical form is equivalent to:

```
int  ioctl ( int  fd ,
            unsigned long  cmd ,
            CMD_DATA_TYPE  * cmddata );
```

Among them, CMD_DATA_TYPE changes with the change of parameter cmd. The detailed description of these three parameters is shown in Table 1-1.

Table 1-1 The three parameters of the ioctl function

parameter	describe	input
name parameter	describe	Output input
name Fd	Framebuffer device file descriptor, which is opened by	Output enter

calling the open function

Cmd	The main cmd (command control words) are as follows: FBIOGET_VSCREENINFO [#21-fbioget_vscreeninfo] : Get screen variable information FBIOPUT_VSCREENINFO [#22-fbioput_vscreeninfo] : Set screen variable information FBIOGET_FSCREENINFO [#23-fbioget_fscreeninfo] : Get screen [#33-fbioget_show] fixed information FBIOPAN_DISPLAY [#24-fbiopan_display] : Set [#32-fbioset_screen_location] PAN display [#31-fbioget_screen_location] : get the overlay display state FBIOSET_SHOW [#34-fbioset_show] : set the overlay display state FBIOGET_GLOBAL_ALPHA [#35-fbioget_global_alpha] : get the overlay alpha attribute FBIOSET_GLOBAL_ALPHA [#36-fbioset_global_alpha] : set the overlay alpha attribute FBIOGET_COLORKEY [#37-fbioget_colorkey] : get the overlay's Colorkey attribute FBIOSET_COLORKEY [#38-fbioset_colorkey] : set the overlay's Colorkey attribute FBIOGET_DISPLAYLAYER_ATTRIBUTES [#39-fbioget_displaylayer_attributes] : [310-fbioget_displaylayer_attributes] get the overlay attribute : set overlay properties [#32-fbioset_screen_location] [#33-fbioget_show] [#34-fbioset_show] [#35-fbioget_global_alpha] [#36-fbioset_global_alpha] [#37-fbioget_colorkey] [#38-fbioset_colorkey] [#310-fbioget_displaylayer_attributes] [#39-fbioset_displaylayer_attributes] FBIOGET_CURSOR_ATTRIBUTE [#41-fbioget_cursor_attribute] : Get mouse layer attributes FBIOSET_CURSOR_ATTRIBUTE [#42-fbioset_cursor_attribute] : Set mouse layer attributes	enter
cmddata	The data types corresponding to each cmd are: Get or set screen variable information: struct fb_var_screeninfo [#612-struct-fb_var_screeninfo] *Type to get screen fixed information: struct fb_fix_screeninfo [#613-struct-fb_fix_screeninfo] *Type to set PAN display: struct fb_var_screeninfo [#612-struct-fb_var_screeninfo] *Type	input Output
parameter name	describe	input Output

to get or set screen overlay coordinate origin:
[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t] *Type
 Get or set the overlay display state: MI_BOOL * type
 to get or set the overlay Alpha: [MI_FB_GlobalAlpha_t](#) [#623-mi_fb_globalalpha_t] * type
 to get or set the overlay Colorkey: [MI_FB_ColorKey_t](#) [#624-mi_fb_colorkey_t] * type
 to get or set the overlay attribute: [MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t] * type
 to get or set the mouse layer attribute: [MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t] *

2. Standard Features

2.1. FBIOGET_VSCREENINFO

- Function

Get the variable information of the screen.

- grammar

```
int ioctl (int fd,FBIOGET_VSCREENINFO,struct fb_var_screeninfo
*var);
```

- describe

Use this interface to obtain variable information of the screen, mainly including resolution and pixel format.

See [struct fb_var_screeninfo](#) [#612-struct-fb_var_screeninfo] for a detailed description of the information .

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOGET_VSCREENINFO [#21-fbioget_vscreeninfo]	ioctl number	enter
var	variable information structure pointer	output

- return value
 - 0 success.
 - -1 fails.

- rely

Header file: fb.h

- Related interface

[FBIOPUT_VSCREENINFO](#) [#22-fbioput_vscreeninfo]

2.2. FBIOPUT_VSCREENINFO

- Function

Set the framebuffer's screen resolution and pixel format, etc.

- grammar

```
int ioctl (int fd, FBIOPUT_VSCREENINFO, structfb_var_screeninfo *var);
```

- describe

Use this interface to set screen resolution and pixel format.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOPUT_VSCREENINFO [#22-fbioput_vscreeninfo]	ioctl number	enter
var	variable information structure pointer	enter

- return value
 - 0 success.
 - -1 fails

- rely

Header file: fb.h

- Notice

The sum of the actual resolution and the offset must be within the virtual resolution range, otherwise the system will automatically adjust the size of the actual resolution to be within the virtual resolution range.

- Related interface

[FBIOGET_VSCREENINFO](#) [#21-fbioget_vscreeninfo]

2.3. FBIOGET_FSCREENINFO

- Function

Get the fixed information of the Framebuffer.

- grammar

```
int ioctl (int fd,FBIOGET_FSCREENINFO, *fix);
```

- describe

Use this interface to obtain Framebuffer fixed information, including video memory starting physical address, video memory size and line spacing. For a detailed description of the information see [struct fb_fix_screeninfo](#) [#613-struct-fb_fix_screeninfo]

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOGET_FSCREENINFO [#23-fbioget_fscreeninfo]	ioctl number	enter
fix	Fixed information structure pointer	output

- return value
 - 0 success
 - -1 fails
- rely

Header file: fb.h

2.4. FBIOPAN_DISPLAY

- Function

Sets the display to start at a different offset from the virtual resolution.

- grammar

```
int ioctl (int fd, FBIOPAN_DISPLAY, *var);
```

- describe

Use this interface to set the display from a different offset from the virtual resolution, leaving the actual resolution unchanged. As shown in Figure 2-1: (xres_virtual, yres_virtual) is the virtual resolution, (xres, yres) is the actual display resolution, and (xoffset, yoffset) is the display offset.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOPAN_DISPLAY [#24-fbiopan_display]	ioctl number	enter
var	variable information structure pointer	enter

- return value
 - 0 success.
 - -1 fails

- rely

Header file: fb.h

- Notice

It must be ensured that the sum of the actual resolution and the offset is within the virtual resolution range, otherwise the setting will fail.

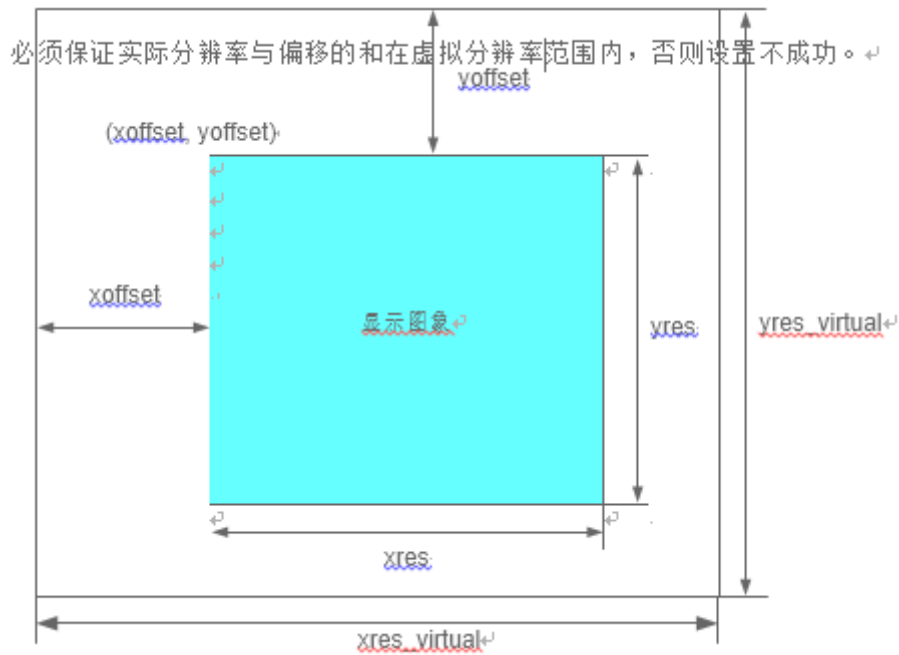


Figure 2-1 Setting display from different offsets in virtual resolution

3. Extended functions

3.1. FBIOGET_SCREEN_LOCATION

- Function

Get the display area of the overlay on the screen

- grammar

```
int ioctl (int fd, FBIOGET_SCREEN_LOCATION, MI_FB_Rectangle_t*
pstRectangle);
```

- describe

Use this interface to get the display area of the overlay on the screen

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOGET_SCREEN_LOCATION [#31-fbioget_screen_location]	ioctl number	enter
pstRectangle	MI_FB_Rectangle_t [#625-mi_fb_rectangle_t] structure pointer, including (x, y) coordinates and width, height. Represents the display area of the overlay on the screen	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_SCREEN_LOCATION](#) [#32-fbioset_screen_location]

3.2. FBIOSET_SCREEN_LOCATION

- Function

Sets the display area of the overlay on the screen

- grammar

```
int ioctl (int fd,FBIOSET_SCREEN_LOCATION,MI_FB_Rectangle_t*
pstRectangle);
```

- describe

Use this interface to set the display area of the overlay on the screen

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSET_SCREEN_LOCATION [#32-fbioset_screen_location]	ioctl number	enter
pstRectangle	MI_FB_Rectangle_t [#625-mi_fb_rectangle_t] structure pointer, including (x, y) coordinates and width, height. Represents the display area of the overlay on the screen	enter

- return value

- 0 success
- -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOGET_SCREEN_LOCATION](#) [#31-fbioget_screen_location]

3.3. FBIOGET_SHOW

- Function

Get the display state of the current overlay.

- grammar

```
int ioctl (int fd, FBIOGET_SHOW, MI_BOOL *bShow);
```

- describe

Use this interface to get the current overlay display state.

- formal parameter

parameter name	describe	input Output
Fd	Framebuffer device file descriptor	enter
FBIOGET_SHOW [#33-fbioget_show]	ioctl number	enter
bShow	Indicates the state of the current overlay: *bShow = MS_TRUE: the current overlay is shown *bShow = MS_FALSE: the current overlay is hidden	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_SHOW](#) [#34-fbioset_show]

3.4. FBIOSET_SHOW

- Function

Show or hide this overlay.

- grammar

```
int ioctl (int fd,FBIOSET_SHOW,MI_BOOL *bShow);
```

- describe

Use this interface to set the overlay display state: shown or hidden.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSSET_SHOW [#34-fbioset_show]	ioctl number	enter
bShow	The display state of this overlay: *bShow = MS_TRUE: show the current overlay *bShow = MS_FALSE: hide the current overlay	enter

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOGET_SHOW](#) [#33-fbioget_show]

3.5. FBIOGET_GLOBAL_ALPHA

- Function

Gets the overlay alpha setting.

- grammar

```
int ioctl (int
fd,FBIOGET_GLOBAL_ALPHA,MI_FB_GlobalAlpha_t*pstAlpha);
```

- describe

Use this interface to get the alpha setting of the current overlay.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOGET_GLOBAL_ALPHA [#35-fbioget_global_alpha]	ioctl number	enter
pstAlpha	MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] structure pointer	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_GLOBAL_ALPHA](#) [#36-fbioset_global_alpha]

3.6. FBIOSET_GLOBAL_ALPHA

- Function

Sets the alpha of the overlay.

- grammar

```
int ioctl (int
fd, FBIOSET_GLOBAL_ALPHA, MI_FB_GlobalAlpha_t*pstAlpha);
```

- describe

Use this interface to set the alpha function of the current overlay.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSET_GLOBAL_ALPHA [#36-fbioset_global_alpha]	ioctl number	enter
pstAlpha	MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] structure pointer	enter

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOGET_GLOBAL_ALPHA](#) [#35-fbioget_global_alpha]

3.7. FBIOGET_COLORKEY

- Function

Get the colorkey of the overlay.

- grammar

```
int ioctl (int fd,FBIOGET_COLORKEY,MI_FB_ColorKey_t*
pstColorKey);
```

- describe

Use this interface to get the colorkey of the overlay.

- formal parameter

parameter name	describe	input Output
Fd	Framebuffer device file descriptor	enter
FBIOGET_COLORKEY [#37-fbioget_colorkey]	ioctl number	enter
pstColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t] structure pointer	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_COLORKEY](#) [#38-fbioset_colorkey]

3.8. FBIOSET_COLORKEY

- Function

Sets the colorkey of the overlay.

- grammar

```
int ioctl (int fd,FBIOSET_COLORKEY,MI_FB_ColorKey_t*
pstColorKey);
```

- describe

Use this interface to set the colorkey function of the current overlay.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSET_COLORKEY [#38-fbioset_colorkey]	ioctl number	enter
pstColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t] structure pointer	enter

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOGET_COLORKEY](#) [#37-fbioget_colorkey]

3.9. FBIOSET_DISPLAYLAYER_ATTRIBUTES

- Function

Set layer information.

- grammar

```
int ioctl ( int fd , FBIOSET_DISPLAYLAYER_ATTRIBUTES ,  
MI_FB_DisplayLayerAttr_t * pstLayerInfo );
```

- describe

This interface is used to set layer information, including the display area of the layer on the screen, canvas resolution, video memory resolution, screen display resolution, and whether to enable premultiplication. [See the description of MI_FB_DisplayLayerAttr_t \[#626-mi_fb_displaylayerattr_t\]](#) for a more detailed description of the above information .

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSET_DISPLAYLAYER_ATTRIBUTES [#39-fbioset_displaylayer_attributes]	ioctl number	enter
pstLayerInfo	MI_FB_DisplayLayerAttr_t [#626-mi_fb_displaylayerattr_t] structure pointer	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#) [#39-fbioset_displaylayer_attributes]

3.10. FBIOGET_DISPLAYLAYER_ATTRIBUTES

- Function

Get layer information.

- grammar

```
int ioctl ( int fd ,  
  
F BIOGET_DISPLAYLAYER_ATTRIBUTES ,  
  
MI_FB_DisplayLayerAttr_t * pstLayerInfo );
```

- describe

Used to obtain layer information, including the display area of the layer on the screen, the video memory resolution, the screen display resolution, and whether premultiplication is enabled.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
F BIOGET_DISPLAYLAYER_ATTRIBUTES [#310- fbioget_displaylayer_attributes]	ioctl number	enter
pstLayerInfo	Layer information structure pointer	output

- return value

- 0 success
- -1 fails

- rely

Header file: mstarFb.h

- Related interface

[F BIOGET_DISPLAYLAYER_ATTRIBUTES](#) [#310-
fbioget_displaylayer_attributes]

4. Mouse related functions

4.1. FBIOGET_CURSOR_ATTRIBUTE

- Function
Get mouse layer information.
- grammar

```
int ioctl ( int fd ,  
  
FBIOGET_CURSOR_ATTRIBUTE ,  
  
MI_FB_CursorAttr_t * pstCursorAttr )
```

- describe
Get the mouse layer information, including **hotspot** , **位置** , **Alpha** , **Colorkey** , whether it is visible.
- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOGET_CURSOR_ATTRIBUTE [#41-fbioget_cursor_attribute]	ioctl number	enter
pstCursorAttr	MI_FB_CursorAttr_t [#6211-mi_fb_cursorattr_t] structure pointer	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[FBIOSET_CURSOR_ATTRIBUTE](#) [#42-fbioset_cursor_attribute]

4.2. FBIOSET_CURSOR_ATTRIBUTE

- Function

Set the mouse layer information.

- grammar

```
int ioctl ( int fd ,  
  
FBIOSET_CURSOR_ATTRIBUTE ,  
  
MI_FB_CursorAttr_t * pstCursorAttr )
```

- describe

Set the mouse layer information, including hotspot, position, Alpha, Colorkey, whether it is visible or not.

- formal parameter

parameter name	describe	input Output
fd	Framebuffer device file descriptor	enter
FBIOSET_CURSOR_ATTRIBUTE [#42-fbioset_cursor_attribute]	ioctl number	enter
pstCursorAttr	MI_FB_CursorAttr_t [#6211-mi_fb_cursorattr_t] structure pointer	output

- return value
 - 0 success
 - -1 fails

- rely

Header file: mstarFb.h

- Related interface

[F BIOGET_CURSOR_ATTRIBUTE](#) [#41-fbioget_cursor_attribute]

5. Error code

Table 2-2 lists all possible error codes that may occur when the function return value is less than 0. These error codes come from standard linux error code definitions. For details, please refer to errno_base.h of the linux kernel source code. The error code can be viewed by printing the Linux standard error code errno, or printing the error message with strerror(errno).

Table 2-2 Error codes

error code	macro definition	describe
1	EPERM	This operation is not supported
12	ENOMEM	Not enough memory
14	EFAULT	The incoming parameter pointer address is invalid
twenty two	EINVAL	Invalid parameter passed in

6. FB data type

6.1. Data types defined in the standard

6.1.1. struct fb_bitfield

- illustrate
Bitfield information for setting the pixel format.
- definition

```
struct fb_bitfield
{
    u32 offset ; /* beginning of bitfield */
    u32 length ; /* length of bitfield */
    u32 msb_right ; /* != 0: Most significant bit is right
*/
} ;
```

- member

member name	describe	Support situation
offset	Color component start bit.	support.
length	The length in bits occupied by the color components.	support.
msb_right	Whether the bit on the right is the most significant bit.	Only this bit is 0, that is, the leftmost bit is the most significant bit.

6.1.2. struct fb_var_screeninfo

- illustrate
Variable screen information.
- definition

```
struct fb_var_screeninfo
{
    u32 xres ; /* visible resolution */

    u32 yres ;

    u32 xres_virtual ; /* virtual resolution */

    u32 yres_virtual ;

    u32 xoffset ; / * offset from virtual to visible */

    u32 yoffset ; /* resolution */

    u32 bits_per_pixel ; / * guess what */

    u32 grayscale ; /* != 0 Graylevels instead of colors */

    struct fb_bitfield red ; /* bitfield in fb mem if
    true color, */ struct fb_bitfield green ; /* else
    only length is significant */ struct fb_bitfield
    blue ;

    struct fb_bitfield transp ; /* transparency */

    u32 nonstd ; / * != 0 Non standard pixel format */

    u32 activate ; /* see FB_ACTIVATE_* */

    u32 height ; /* height of picture in mm */

    u32 width ; /* width of picture in mm */

    u32 accel_flags ; / * (OBSOLETE) see fb_info.flags */

    /* Timing: All values in pixclocks, except pixclock (of
    course) */

    u32 pixclock ; / * pixel clock in ps (pico seconds) */

    u32 left_margin ; / * time from sync to picture */

    u32 right_margin ; /* time from picture to sync */

    u32 upper_margin ; / * time from sync to picture */
}
```



```
    u32 lower_margin ;

    u32 hsync_len ; /  * length of horizontal sync */

    u32 vsync_len ; /  * length of vertical sync */

    u32  sync ; /* see FB_SYNC_* */

    u32  vmode ; /* see FB_VMODE_* */

    u32  rotate ; /* angle we rotate counter clockwise */

    u32  reserved [ 5 ]; /* Reserved for future compatibility
*/

} ;
```

• member

member name	describe	Support situation
xres	Visible screen width (pixels)	support
yres	Visible screen height (pixels)	support
xres_virtual	Virtual screen width (image width in video memory), when the value is less than xres, xres will be modified to make the xres value equal to this value	support
yres_virtual	Virtual screen height (image height in video memory), when the value is less than yres, yres will be modified to make the yres value equal to this value. Combined with xres_virtual, it can be used to quickly pan the image horizontally or vertically.	support
xoffset	number of offset pixels in the x direction	support, default is 0
yoffset	the number of offset pixels in the y direction	support, default is 0
bits_per_pixel	bits per pixel	support
member name	describe	Support situation

grayscale	grayscale	Not supported, the default value is 0, which means color
red	Bitfield information for red in the color component	support
green	Bitfield information for green in the color component	support
blue	Bitfield information for blue in color components	support
transp	Bitfield information for the alpha component of the color components	support
nonstd	Is it a standard pixel format	Not supported, the default value is 0, which means that the standard pixel format is supported.
activate	The time when the setting takes effect	Not supported, the default value is FB_ACTIVATE_NOW, which means the setting takes effect immediately.
height	Screen height, in mm	Not supported, the default value is -1.
width	Screen width in mm	Not supported, the default value is -1.
accel_flags	Speed up sign	Not supported, the default value is -1.
pixclock	Display the time required for a point, in ns	Not supported, the default value is -1.
member name left_margin right_margin hsync_len	describe They are the left blanking signal, the right blanking signal, and the horizontal synchronization duration. The sum of these	Support situation Not supported, default value is 64

nsync_len	synchronization duration. The sum of these three values is equal to the horizontal retrace time, and the unit is dot clock.	
upper_margin	They are the upper blanking signal, the lower	Not supported, the default value is -1.
lower_margin	blanking signal, and the vertical	
vsync_len	synchronization duration. The sum of these three values is equal to the vertical retrace time, and the unit is dot clock.	

| sync | Synchronization signal mode | Not supported, the default value is -1.

|| vmode | scan mode | Not supported, default -1. || rotate | Angle of clockwise rotation | Not supported, the default value is 0, which means no rotation. |

- Precautions

The default resolution of the high-definition device graphics layer is 1280x720; the default resolution of the SD device graphics layer is 720x576, and the default resolution of the mouse layer is 128x128. The pixel format is ARGB1555.

- Related data types and interfaces

[struct fb_bitfield](#) [#611-struct-fb_bitfield]

[FBIOGET_VSCREENINFO](#) [#21-fbioget_vscreeninfo]

[FBIOPUT_VSCREENINFO](#) [#22-fbioput_vscreeninfo]

6.1.3. struct fb_fix_screeninfo

- illustrate

Fixed screen info.

- definition

```
struct fb_fix_screeninfo
{
    char id [ 16 ]; /* identification string eg "TT Builtin"
    */ unsigned long smem_start ; /* Start of frame buffer
    mem (physical
```

```

address) */

    u32 smem_len ; / * Length of frame buffer mem */

    u32 type ; /* see FB_TYPE_* */

    u32 type_aux ; / * Interleave for interleaved Planes */

    u32 visual ; /* see FB_VISUAL_* */

    u16 xpanstep ; /* zero if no hardware panning */

    u16 ypanstep ; /* zero if no hardware panning */

    u16 ywrapstep ; /* zero if no hardware ywrap */

    u32 line_length ; / * length of a line in bytes */
    unsigned long mmio_start ;
    /* Start of Memory Mapped I/O (physical
address) */

    u32 mmio_len ; / * Length of Memory Mapped I/O */

    u32 accel ; /* Indicate to driver which specific
chip/card we have */

    u16 reserved [ 3 ]; /* Reserved for future compatibility
*/

} ;

```

- member

member name	describe	Support situation
id	Device driver name.	support.
smem_start	The starting physical address of the video memory.	support.
smem_len	Video memory size.	support.
member type name	describe Graphics card type.	Support situation Fixed to FB_TYPE_PACKED_PIXELS,

		indicating that the pixel values are closely packed.
type_aux	Additional type.	Not supported, meaningless under FB_TYPE_PACKED_PIXELS graphics card type.
visual	color mode.	Not supported, the default is FB_TYPE_PACKED_PIXELS. No meaning under the graphics card type.
xpanstep	Horizontal PAN display is supported: 0: Not supported. Non-0: Supported, this value is used to represent the pixel value of each step in the horizontal direction.	Fixed to 1.
ypanstep	PAN display in vertical orientation is supported: 0: Not supported. Non-0: Supported, this value is used to represent the pixel value per step in the vertical direction.	Fixed to 1.
ywrapstep	This method is similar to ypanstep, the difference is that when it is displayed to the bottom, it can return to the beginning of the video memory for display.	Not supported, defaults to 0.
line_length	Bytes per line.	support.
mmio_start	Memory mapped I/O first address.	Not supported, defaults to 0.
mmio_len	Memory mapped I/O length.	Not supported, defaults to 0.
accel_member_name	Displays supported hardware acceleration devices.	Not supported, default is FB_ACCEL_NONE, no acceleration device.

reserved

reserve.

Not supported, the default value is 0.

- Related data types and interfaces

[FBIOGET_FSCREENINFO](#) [#23-fbioget_fscreeninfo]

6.2. Extended data types

6.2.1. MI_FB_ColorFmt_e

- illustrate

Collection of pixel formats supported by MStarFB.

- definition

```
typedef enum
{
    E_MI_FB_COLOR_FMT_RGB565    = 1 ,
    E_MI_FB_COLOR_FMT_ARGB4444  = 2 ,
    E_MI_FB_COLOR_FMT_ARGB8888  = 5 ,
    E_MI_FB_COLOR_FMT_ARGB1555  = 6 ,
    E_MI_FB_COLOR_FMT_YUV422    = 9 ,
    /// Invalid color format .
    E_MI_FB_COLOR_FMT_INVALID    = 12 ,
} MI_FB_ColorFmt_e ;
```

- member

member name	describe
E_MI_FB_COLOR_FMT_RGB565	RGB565 format.
E_MI_FB_COLOR_FMT_ARGB4444	ARGB444 format.
E_MI_FB_COLOR_FMT_ARGB8888	ARGB8888
E_MI_FB_COLOR_FMT_ARGB1555	ARGB1555
E_MI_FB_COLOR_FMT_YUV422	YUV422
E_MI_FB_COLOR_FMT_INVALID	invalid pixel format

- Related data types and interfaces

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

6.2.2. MI_FB_DisplayLayerAttrMaskbit_e

- illustrate

Identify which members of the [MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t] structure are updated

- definition

```
typedef enum
{
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS    = 0x1 ,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE   = 0x2 , _
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_BUFFER_SIZE = 0x4 , _
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE = 0x8 , _
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_PREMUL      = 0x10 , _
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_COLOR_FMB   = 0x20 , _
}
```

```
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_OUTPUT_COLORSPACE = 0x40 ,  
    -  
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DST_DISP = 0x80 , -  
  
} MI_FB_DisplayLayerAttrMaskbit_e ;
```

• member

member name	describe
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS	Overlay position changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE	The width or height of the overlay's display area on the screen changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_BUFFER_SIZE	Display resolution changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE	Screen resolution changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_PREMUL	Whether the premultiplication property has changed
E_MI_FB_DISPLAYLAYER_ATTR_MASK_COLOR_FMB	FB's Color format property changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_OUTPUT_COLORSPACE	The Color Space of the overlay output changes
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DST_DISP	The overlay output to the display chain target module changes

- Related data types and interfaces

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.3. MI_FB_GlobalAlpha_t

- illustrate

Alpha information structure

- definition

```
typedef struct MI_FB_GlobalAlpha_s
{
    MI_BOOL  bAlphaEnable ; /* alpha enable flag */

    MI_BOOL  bAlphaChannel ; /* alpha channel enable flag */

    MI_BOOL  bAlphaInvert ; /* alpha invert flag */

    MI_U8    u8Alpha0 ; /* alpha0 value */

    MI_U8    u8Alpha1 ; /* alpha1 value */

    MI_U8    u8GlobalAlpha ; /* global alpha value */  MI_U8
    u8Reserved ;

} MI_FB_GlobalAlpha_t ;
```

- member

member name	describe
bAlphaEnable	Alpha overlay is enabled, the default is 1.
bAlphaChannel	Alpha channel enabled, default is 0.
bAlphaInvert	Alpha inversion flag, used under argb1555, 0 is opaque.
u8Alpha0	Alpha0 value, the range is 0 ~ 255, the default is 255. In RGB1:5:5:5 format, when the highest bit is 0, select this value as the Alpha value of Alpha overlay.
u8Alpha1	Alpha1 value, the range is 0 ~ 255, the default is 255. In RGB1:5:5:5 format, when the highest bit is 1, select this value as the alpha value of Alpha overlay.
u8GlobalAlpha	Global Alpha value, the range is 0 ~ 255, the default is 255. Works when alpha channel is enabled.
u8Reserved	reserve

- Precautions
 - Alpha0 value is only valid for cursor, UI's alpha0 is always 0.
 - Alpha Overlay is only done if Alpha Overlay is enabled, otherwise overlays on top will overwrite those below.
 - The formula for calculating the superimposed alpha value is as follows:
 - When the alpha channel is enabled, the global alpha participates in the overlay.
 - For chips that do not support global alpha and pixel alpha overlay, the calculation formula of overlay alpha value is as follows: $\alpha = u * 8 * GlobalAlpha$
 - For chips that support global alpha and pixel alpha overlay, the calculation formula for overlay alpha value is as follows: $\alpha = u * 8 * GlobalAlpha ** \alpha_{pixel}$
 - When the alpha channel is disabled, the superimposed alpha value is equal to the pixel alpha value, namely: $\alpha = \alpha_{pixel}$

- Related data types and interfaces

[FBIOGET_GLOBAL_ALPHA](#) [#35-fbioget_global_alpha]

[FBIOSET_GLOBAL_ALPHA](#) [#36-fbioset_global_alpha]

6.2.4. MI_FB_ColorKey_t

- illustrate

Colorkey information structure, used for property setting of colorkey.

- definition

```
typedef struct MI_FB_ColorKey_s
{
    MI_BOOL    bKeyEnable ;

    MI_U8      u8Red ;

    MI_U8      u8Green ;

    MI_U8      u8Blue ;

} MI_FB_ColorKey_t
```

- member

member	describe
bKeyEnable	Whether Colorkey is enabled or not. TRUE: enable; FALSE: disable.
u8Red	Colorkey's R component value
u8Green	Colorkey's G component value
u8Blue	Colorkey's B component value

- Related data types and interfaces

[FBIOGET_COLORKEY](#) [#37-fbioget_colorkey]

[FBIOSET_COLORKEY](#) [#38-fbioset_colorkey]

6.2.5. MI_FB_Rectangle_t

- illustrate

[MI_FB_Rectangle_t](#) [#625-mi_fb_rectangle_t] structure, used to describe the display area of the overlay

- definition

```
typedef struct MI_FB_Rectangle_s
{
    MI_U32  u32Xpos ;

    MI_U32  u32Ypos ;

    MI_U32  u32Width ;

    MI_U32  u32Height ;

} MI_FB_Rectangle_t ;
```

- member

member	describe
u32Xpos	top left x coordinate
u32Ypos	upper left corner y coordinate
u32Width	width
u32Height	high

- Related data types and interfaces

[FBIOGET_SCREEN_LOCATION](#) [#31-fbioget_screen_location]

[FBIOSET_SCREEN_LOCATION](#) [#32-fbioset_screen_location]

6.2.6. MI_FB_DisplayLayerAttr_t

- illustrate

The [MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t] structure describes the attribute information of the overlay.

- definition

```
typedef struct MI_FB_DisplayLayerAttr_s
{
    MI_U32  u32Xpos ;    /**the x pos of origin point in screen*/
    MI_U32  u32YPos ;    /**the y pos of origin point in screen*/
    MI_U32  u32dstWidth ; /**display buffer dest width in
screen*/
    MI_U32  u32dstHeight ; /**display buffer dest height in
screen*/
    MI_U32  u32DisplayWidth ; /**the width of display buf in
fb */
    MI_U32  u32DisplayHeight ; /**the height of display buf in
fb. */
    MI_U32  u32ScreenWidth ; /**the width of screen */
    MI_U32  u32ScreenHeight ; /** the height of screen */
    MI_BOOL  bPreMul ; /**the data drawn in buffer whether is
premultiply alpha
or not*/
    MI_FB_ColorFmt_e  eFbColorFmt ; /**the color format of
framebuffer*/
    MI_FB_OutputColorSpace_e  eFbOutputColorSpace ; /**output
color space*/
    MI_FB_DstDisplayplane_e  eFbDestDisplayPlane ;
/**destination
displayplane*/
}
```

```
MI_U32 u32SetAttrMask ; /** display attribute modify
mask*/

} MI_FB_DisplayLayerAttr_t ;
```

• member

member	describe
u32Xpos	The abscissa of the origin of the layer on the screen.
u32YPos	The ordinate of the origin of the layer on the screen.
u32dstWidth	The display width of the layer on the screen
u32dstHeight	The display height of the layer on the screen
u32DisplayWidth	The width of the memory resolution.
u32DisplayHeight	High memory resolution.
u32ScreenWidth	The width of the screen display resolution.
u32ScreenHeight	The screen displays high resolution.
bPreMul	Whether the data in the FB is premultiplied data.
eFbColorFmt	FB's pixel format.
u32SetAttrMask	The parameter modifies the mask bit when setting the layer information.
eFbOutputColorSpace	Sets the Color space of the overlay output
eFbDestDisplayPlane	Sets the destination module of the overlay output to the display chain. The default is E_MI_FB_DST_OP0.

• Related data types and interfaces

[MI_FB_DisplayLayerAttrMaskbit_e](#) [#622-mi_fb_displaylayerattrmaskbit_e]

[MI_FB_ColorFmt_e](#) [#621-mi_fb_colorfmt_e]

[MI_FB_DstDisplayplane_e](#) [#628-mi_fb_dstdisplayplane_e]

[MI_FB_OutputColorSpace_e](#) [#627-mi_fb_outputcolorspace_e]

6.2.7. MI_FB_OutputColorSpace_e

- illustrate

Color Space for the MStarFB overlay output.

- definition

```
typedef enum
{
    E_MI_FB_OUTPUT_RGB    = 0 ,
    E_MI_FB_OUTPUT_YUV    = 1
} MI_FB_OutputColorSpace_e
```

- member

member	describe
E_MI_FB_OUTPUT_RGB	RGB ColorSpace
E_MI_FB_OUTPUT_YUV	YUV ColorSpace

- Related data types and interfaces

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.8. MI_FB_DstDisplayplane_e

- illustrate

The MStarFB overlay is output to the destination module of the display chain. By configuring different Display chains, it is possible to choose the image adjustment process that the overlay participates in.

- definition

```
typedef enum
{
    E_MI_FB_DST_OP0 = 0 ,
    E_MI_FB_DST_OP1 = 1 ,
    E_MI_FB_DST_INVALID
} MI_FB_DstDisplayplane_e
```

- member

member	describe
E_MI_FB_DST_OP0	OP path
E_MI_FB_DST_OP1	OP1 path
E_MI_FB_DST_INVALID	MIXER2OP1

- Precautions

This interface is only valid under Ikayaki series chips.

- Related data types and interfaces

[MI_FB_DisplayLayerAttr_t](#) [#626-mi_fb_displaylayerattr_t]

6.2.9. MI_FB_CursorAttrMaskbit_e

- illustrate

Describe which attributes of the [MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t] structure change

- definition

```
typedef enum
{
```



```
E_MI_FB_CURSOR_ATTR_MASK_ICON    = 0x1 ,
E_MI_FB_CURSOR_ATTR_MASK_POS     = 0x2 ,
E_MI_FB_CURSOR_ATTR_MASK_ALPHA   = 0x4 , _
E_MI_FB_CURSOR_ATTR_SHOW         = 0x8 , _
E_MI_FB_CURSOR_ATTR_HIDE         = 0x10 , _
E_MI_FB_CURSOR_ATTR_MASK         = 0x1F _
} MI_FB_CursorAttrMaskbit_e ;
```

- member

member	describe
E_MI_FB_CURSOR_ATTR_MASK_ICON	mouse icon changes
E_MI_FB_CURSOR_ATTR_MASK_POS	mouse position changes
E_MI_FB_CURSOR_ATTR_MASK_ALPHA	Alpha changes
E_MI_FB_CURSOR_ATTR_SHOW	show mouse
E_MI_FB_CURSOR_ATTR_MASK_HIDE	hide the mouse
E_MI_FB_CURSOR_ATTR_MASK	Used to identify all settable properties of the mouse

- Related data types and interfaces

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

6.2.10. MI_FB_CursorImage_t

- illustrate

[MI_FB_CursorImage_t](#) [#6210-mi_fb_cursorimage_t] structure, describing the mouse icon data information.

- definition

```
typedef struct MI_FB_CursorImage_s
{
    MI_U32  u32Width ;  /**width, unit pixel*/
    MI_U32  u32Height ;  /**Height, unit pixel*/
    MI_U32  u32Pitch ;  /**Pitch, unit pixel*/
    MI_FB_ColorFmt_e  ColorFmt ;  /**Color format*/
    const char *  data ;
} MI_FB_CursorImage_t ;
```

- member

member	describe
u32Width	Icon width
u32Height	Icon height
u32Pitch	pitch
eColorFmt	Icon Data Pixel Format
data	Icon data

- Precautions

The maximum value for the width and height of the icon is 128.

- Related data types and interfaces

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t]

[MI_FB_ColorFmt_e](#) [#621-mi_fb_colorfmt_e]

6.2.11. MI_FB_CursorAttr_t

- illustrate

[MI_FB_CursorAttr_t](#) [#6211-mi_fb_cursorattr_t] structure, describing mouse layer information

- definition

```
typedef struct MI_FB_CursorAttr_s
{
    MI_U32    u32XPos ;
    MI_U32    u32YPos ;
    MI_U32    u32HotSpotX ;
    MI_U32    u32HotSpotY ;
    MI_FB_GlobalAlpha_t  stAlpha ;
    MI_FB_ColorKey_t  stColorKey ;
    MI_BOOL    bShown ;
    MI_FB_CursorImage_t  stCursorImageInfo ;
    MS_U16    u16CursorAttrMask ;
} MI_FB_CursorAttr_t ;
```

- member

member	describe
u32XPos	The abscissa of the mouse position
u32YPos	mouse position ordinate
u32HotSpotX	Mouse hotspot abscissa
u32HotSpotY	The ordinate of the mouse hotspot
stAlpha	MI_FB_GlobalAlpha_t [#623-mi_fb_globalalpha_t] structure, mouse layer alpha information
stColorKey	MI_FB_ColorKey_t [#624-mi_fb_colorkey_t] structure, mouse layer colorkey information
bShown	Whether the mouse is visible TRUE means visible, FALSE means invisible
MI_FB_CursorImage_t [#6210-mi_fb_cursorimage_t]	Icon data information.
u16CursorAttrMask	Set mouse layer parameters to modify mask bits

- Related data types and interfaces

[MI_FB_CursorAttrMaskbit_e](#) [#629-mi_fb_cursorattrmaskbit_e]

[MI_FB_CursorImage_t](#) [#6210-mi_fb_cursorimage_t]

7. Introduction to PROCFS

7.1. cat

- debug info

```

# cat /proc / mi_modules / mi_fb0

##### OnDumpOSDAtr #####
#####
Framebuffer id = MStar FB0
xres = 1280 , yres = 720
xres_virtual = 1280 , yres_virtual = 1440
xoffset = 0 , yoffset = 0
fix . line_length = 0 xa00 Bytes
fix . smem_start = 0xf800000 Memory Size = 0x400000 _
_ Bytes
Gop ID = 0
Gwin ID = 0
Open Count = 1
Visible State = 0
MIU Sel = 0
ColorSpace = YUV
ColorFomrmat = ARGB1555
StretchWindow Pos [ 0 , 0 ]
StretchWindow Src [ 1280 , 720 ], StretchWindow Dst [ 1920 ,
1080 ]
Gwin Pos [ 0 , 0 ]
Gwin Size [ 1280 , 720 ]
Gwin PhyAddr = 0 x0
ColorKey Enable = 0
ColorKey Val = [ 0 , 0 , 0 ]
Enable Alpha Blend = 1
Enalbe Multi Alpha = 0
Global Alpha Val = 0 x0
Alpha0 = 0 x0 , Alpha1 = 0x0 _,( Only for ARGB1555 )
GOP Hstart = 192
Current TimingWidth = 1920 , TimingWidth = 1080 , hstar = 192
##### end
#####

##### OnDumpHwcursorAttr
#####
#####
Cursor Gop ID = 4
Cursor Gwin ID = 6
Cursor MIU Sel = 0
Cursor PhyAddr = 0 x166000
Cursor ColorFmt = ARGB8888
Cursor Icon Width = 44 , Height = 56
Cursor HotSpot [ 18 , 9 ]
Cursor request pos [ 712 , 207 ]
Cursor Visible = 1
Cursor Gwin Pos [ 694 , 198 ]

```

```
Cursor Gwin Size [ 44 , 56 ]
Cursor Gwin Pitch = 0 x200 Bytes Curosr StretchWindow pos [
0 , 0 ] Cursor StretchWin Src [ 1280 , 720 ], Dst [ 1920 , 1080
] Cursor

ColorKey Enable = 0
Cursor ColorKey Value = [ 0 xff , 0 xff , 0 xff ]
##### OnDumpHwcursorAttr ###
#####
```

- Analysis of debug information
- Record the current usage of Fbdev && Hwcursor, as well as OSD layer properties, OSD device properties, and Hwcursor properties, which can be dynamically obtained to facilitate debugging and testing.
- Parameter Description

parameter		describe
OSD info	framebuffer id	Framebuffer device ID
	xres	Visible resolution width
	yres	Visible resolution width and height
	xres_virtual	width of virtual resolution
	yres_virtual	height of virtual resolution
	xoffset	x offset from visible resolution to virtual resolution
	yoffset	y offset from visible resolution to virtual resolution
	fix.line_length	virtual resolution bytes per line
	fix.smem_start	starting address in the framebuffer
	Memory Size	Frame buffer memory length, in bytes
OSD device	Gop ID	Layer hardware ID

info	Gwin ID	Layer hardware display window ID
	Open Count	The number of times this graphics layer has been opened. Incremented by 1 when the user calls open(); decremented by 1 when close() is called
	Visible State	The layer shows the status. Value: {1 means visible, 0 means invisible} User display calls FBIOSET_SHOW to set the visible state of the layer.
	MIU Sel	The number of the physical memory where the framebuffer data is stored. For MSR620 devices this value is 0
	ColorSpace	The color space for the output of this graphics layer. Value: {YUV means output YUV color space; RGB means output RGB color space }. Default is YUV
	ColorFomrmat	The value of the graphics layer format : {ARGB1555, ARGB4444, ARGB8888, RGB565}. Update after user sets format item in variable screen info
	StretchWindow Pos	The starting position of the graphics layer on the display device. Unit: pixel defaults to 0, the user can call FBIOSET_SCREEN_LOCATION to set
	StretchWindow Src	The width and height of the graphics layer's native resolution. Units: Pixels are amplified to the current timing by the hardware zoom function. Update after user sets visible resolution in variable screen info
	StretchWindow Dst	The width and height of the graphics layer on the display device. The default setting is the output timing size. Can be set by FBIOSET_SCREEN_LOCATION
	Gwin Pos	The starting position unit of the graphics layer display window

	: pixels For OSD layers, the value is always (0,0)
Gwin Size	The graphics layer displays the width and height of the window. Units: Pixels For OSD layers, this value is always set to the width and height of the visible resolution. Updated after the user sets the visible resolution in the variable screen info.
Gwin PhyAddr	The physical address of the current display data of the graphics layer
ColorKey Enable	Whether the ColorKey of the graphics layer is enabled . Value: {0 means Disable ColorKey; 1 means Enable ColorKey} User settings can be set through FBIOSET_COLORKEY
ColorKey Val	The value of the RED, GREEN, BLUE channels of the ColorKey of the graphics layer : the value of each channel ranges from 0 to 255. The value of ColorKey val is expressed in RGB888 format, and the user can update it after setting it through FBIOSET_COLORKEY
Enable Alpha Blend	Whether the graphics layer alpha is enabled. Value: {0 means no, 1 means yes}, the default is 1. Users can update after setting through FBIOSET_GLOBAL_ALPHA. If this option is off, the pixel alpha configuration does not take effect. When this option is turned on and Enable Multi Alpha is turned off, only pixel alpha takes effect (for ARGB1555 format, pixels whose Alpha bit is 1 use Global Alpha as pixel alpha, and pixels whose Alpha bit is 0 use 0xff as pixel alpha). When the item is turned on and Enable Multi Alpha is in effect, both pixel alpha and Global Alpha are in effect
Enable Multi Alpha	Whether the global alpha is valid or not The switch value is {0, 1 no} The default value is 0 Enable Multi Alpha, Global Alpha will take effect

	Global Alpha Val	Global Alpha
	Alpha0	In ARGB1555 format, when the highest bit is 0, select Alpha0 as the Alpha value of Alpha blending : 0~255 The default is 0
	Alpha1	In ARGB1555 format, when the highest bit is 1, select Alpha1 as the Alpha value of Alpha blending : 0~255 The default is 0
	GOP Hstart	The offset of the layer hardware relative to the display window. Value: This value is related to the current output timing. When the output timing changes, FBIOSET_DISPLAYLAYER_ATTRIBUTES and the u32SetAttrMask value of the input parameter MI_FB_DisplayLayerAttr_s include the E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE mask bit to notify the layer driver that the output timing changes. The layer driver will set the corresponding hstart value according to the current timing
	Current TimingWidth,	The width and height of the current output timing and the offset to the display window that should be set to the graphics layer hardware under the current timing. This item is used in conjunction with GOP Hstart to check whether the hstart value set to the hardware layer is consistent with the current timing.
	Current TimingHeight,	
	hstar	
Hwcursor info	Cursor Gop ID	Mouse Layer Hardware ID
	Cursor Gwin ID	Mouse layer hardware display window ID
	Cursor MIU Sel	The number of the physical memory used to save the mouse icon data. For MSR620 devices this value is 0
	Cursor PhyAddr	The physical address of the current display data of the mouse layer

Cursor ColorFmt	Mouse Layer Format Value : {ARGB1555, ARGB4444, ARGB8888, RGB565}. User can set through FBIOSET_CURSOR_ATTRIBUTE
Cursor Icon Width	The width and height of the mouse icon. Unit: Pixels can be set by FBIOSET_CURSOR_ATTRIBUTE
Cursor HotSpot	Hotspot information unit of the mouse icon: pixel Hotspot information is usually related to the mouse icon, and the user can set it through FBIOSET_CURSOR_ATTRIBUTE
Cursor request pos	The position information of the mouse icon Unit : pixel user set by FBIOSET_CURSOR_ATTRIBUTE
Cursor Visible	Whether the mouse layer is visible Value : {0, means invisible, 1 means visible} User set through FBIOSET_CURSOR_ATTRIBUTE
Cursor Gwin Pos	The starting position of the mouse layer display window Unit : pixel
Cursor Gwin Pitch	The number of bytes per row of the mouse layer. For mouse layers, the value is fixed at 0x200
Curosr StretchWindow pos	The starting position of the mouse layer on the display device. Unit: pixel
Cursor StretchWin Src	Original size of mouse layer Unit: pixel. This value is usually the same as the OSD display resolution.
Cursor StretchWin Dst	The width and height of the mouse layer on the display device This value is consistent with the output timing.
Cursor ColorKey Enable	Whether the ColorKey of the mouse layer is enabled Value: {0 means Disable ColorKey; 1 means Enable ColorKey}.

User set via FBIOSET_CURSOR_ATTRIBUTE	
Cursor ColorKey Value	The value of the RED, GREEN, BLUE channels of the mouse layer ColorKey : the value of each channel ranges from 0-255. The value of ColorKey val is expressed in RGB888 format, and the user can update it after setting it through FBIOSET_CURSOR_ATTRIBUTE

7.2. echo

Function	Turn the specified layer on or off
Order	echo GUI_SHOW [LayerID] [Status] > /proc/mi_modules/mi_fb0
Parameter Description	[LayerID] Layer ID [Status] on: on; off: off
Example	echo GUI_SHOW 0 on > /proc/mi_modules/mi_fb0 show layer fb0 echo GUI_SHOW 0 off > /proc/mi_modules/mi_fb0 close layer fb0

Function	Turn the hardware mouse on or off
Order	echo CURSOR_SHOW [Status] > /proc/mi_modules/mi_fb0
Parameter Description	[Status] on: open; off: close
Example	echo CURSOR_SHOW on > /proc/mi_modules/mi_fb0 show the mouse icon echo CURSOR_SHOW off > /proc/mi_modules/mi_fb0 close the mouse icon

Function	Set Colorkey for the specified layer
Order	echo GUI_SET_CLRKEY [LayerID] [Red] [Green] [Blue] > /proc/mi_modules/mi_fb0
Parameter Description	[LayerID] Layer ID [Red] Red component in hexadecimal, ARGB888 format [Green] Green component in hexadecimal, ARGB888 format [Blue] Blue component in hexadecimal, ARGB888 format
Example	echo GUI_SET_CLRKEY 0 ff 00 00 > /proc/mi_modules/mi_fb0 Set the Colorkey of fb0 to red

Function	The mouse icon currently displayed by Dump
Order	echo CURSOR_DUMP [Path] >/proc/mi_modules/mi_fb0
Parameter Description	[Path] The path of the dump
Example	echo CURSOR_DUMP /mnt > /proc/mi_modules/mi_fb0 dump cursor Icon to /mnt path, the file name is CursorData.raw, open it with 7yuv, se and height to 128,128, and the format can be displayed according to the format of c

Function	Set the pixel alpha value for alpha bit0 and alpha bi1 of ARGB1555 format respective Because currently msr620 does not support this feature, it has not been implemente
Order	echo GUI_SETALPHA_ARGB1555 [LayerID] [Alpha0] [Alpha1]> /proc/mi_modules/mi
Parameter Description	[LayerID] Layer ID [Alpha0] Value of alpha0 [Alpha1] Value of alpha1
Example	none

